

1-Portada

Nombre del proyecto: MoneyTrack: Aplicación móvil para el control de gastos personales.

Equipo: Arnold Mejía Tineo.

Asignatura: Seminario de Proyecto II.

Clave: ISW-411.

Carrera: Ingeniería de Software.

Universidad: Universidad Abierta para Adultos (UAPA).

Fecha: 15 de julio del 2026.

2-Descripción del problema

Muchas personas tienen problemas para manejar bien sus finanzas personales. Falta una herramienta sencilla que les permita anotar y revisar los gastos del día a día. En muchos casos los gastos se recuerdan a medias, o se anotan en distintos lugares. Esto hace difícil llevar un control preciso del dinero, y también dificulta tomar mejores decisiones financieras.

Por eso surge MoneyTrack. Es una app móvil para Android que permite registrar, consultar, actualizar y eliminar gastos personales. Usa una base de datos SQLite guardada localmente en el dispositivo. Es una solución práctica, fácil de usar, para organizar la información financiera. No depende de internet ni de servicios externos.

3-Objetivos y alcance

Objetivo general

Desarrollar una aplicación móvil para Android que permita a los usuarios registrar y administrar sus gastos personales mediante almacenamiento local en SQLite, ofreciendo una herramienta sencilla para mejorar el control de sus finanzas.

Objetivos específicos

- Implementar un sistema de registro e inicio de sesión para los usuarios.
- Desarrollar un CRUD completo para la gestión de gastos personales.
- Almacenar la información utilizando una base de datos SQLite.
- Mostrar un Dashboard con información resumida de los gastos registrados.
- Aplicar validaciones en los formularios para garantizar la integridad de los datos.

Alcance

La aplicación permite registrar usuarios, iniciar sesión, registrar gastos, consultar la lista de gastos, actualizar registros existentes, eliminar información y visualizar un resumen financiero desde el Dashboard. Todo el funcionamiento se realiza utilizando almacenamiento local mediante SQLite.

Funcionalidades excluidas

Por limitaciones de tiempo y por el alcance definido para esta primera versión, no se implementaron funcionalidades como sincronización en la nube, generación de reportes en PDF, recuperación de contraseña, gráficos estadísticos, control de presupuestos, exportación de datos o sincronización entre dispositivos.

4-Justificación e implicaciones socioeconómicas

MoneyTrack surge de la necesidad de tener una herramienta sencilla para registrar y controlar gastos personales desde el celular. Cualquier usuario puede organizar mejor su información financiera con esta app. Puede saber cuánto ha gastado. Esto ayuda a planificar mejor su economía.

Desde lo educativo, el proyecto me permitió aplicar lo aprendido en la carrera. Desarrollo de apps móviles, programación orientada a objetos, bases de datos SQLite, arquitectura de software. Desde lo social, la app promueve hábitos de organización financiera. Puede contribuir a que las personas manejen sus recursos económicos de forma más responsable.

5-Factibilidad

La factibilidad de MoneyTrack se analizó viendo los aspectos técnicos, el tiempo disponible y los recursos necesarios para hacerlo. Este análisis mostró que el proyecto era viable dentro de las condiciones de la asignatura. Las tecnologías elegidas ya las conocía el desarrollador. Los recursos estaban disponibles. El cronograma alcanzaba para cumplir los objetivos.

Factibilidad técnica

Técnicamente el proyecto era totalmente viable, se usaron tecnologías bien conocidas para hacer apps Android. Se eligió Android Studio como entorno de desarrollo, es la herramienta oficial para crear apps Android, se integra bien con el SDK, el emulador y las herramientas de depuración.

Como lenguaje se usó Java. Fue el lenguaje trabajado en la asignatura. Se integra completamente con Android Studio. Tiene bastante documentación y recursos para aprender.

Para guardar la información se usó SQLite. Es una base de datos local integrada en Android, guarda la información de forma permanente sin necesitar servidor ni internet. Esta elección permitió hacer una app funcional, estable, adecuada para lo que el proyecto necesitaba.

La arquitectura MVC ayudó a organizar el código. Separó la lógica de negocio, la interfaz de usuario y el acceso a los datos. Esto mejoró el mantenimiento y la comprensión del sistema.

Factibilidad de tiempo

El desarrollo se planificó según el cronograma de Seminario de Proyecto II. El trabajo se dividió en varias etapas, análisis del problema, elaboración de documentación, diseño de la base de datos, desarrollo de la app, pruebas funcionales, documentación final.

Cada entrega parcial ayudó a avanzar poco a poco hasta tener una app completamente funcional. La planificación del tiempo permitió implementar las funcionalidades principales sin atrasarse con las fechas de entrega.

Factibilidad de recursos

El proyecto no necesitó infraestructura especializada ni herramientas costosas. Se usó una computadora personal capaz de correr Android Studio, un dispositivo móvil Android y un emulador para las pruebas.

También se usaron herramientas gratuitas como GitHub para el control de versiones y respaldo del código. La documentación oficial de Android Developers, Java y SQLite sirvió de apoyo durante todo el proceso.

Como la app usa almacenamiento local, no hizo falta contratar hosting, servidores web ni bases de datos remotas. Esto bajó bastante los costos de implementación, el proyecto se pudo hacer solo con recursos que el estudiante ya tenía disponibles.

6-Tecnologías utilizadas

Para desarrollar MoneyTrack se escogió un grupo de tecnologías que permitieron construir una app funcional, estable, acorde a lo que pedía el proyecto final, cada herramienta se eligió viendo su compatibilidad con Android, qué tan fácil era de usar, la documentación disponible y si encajaba con el alcance de esta primera versión.

Android Studio

Se usó Android Studio como el IDE principal para construir la app, se eligió porque es el entorno oficial que recomienda Google para desarrollo Android, se integra completo con el SDK, el emulador de dispositivos, herramientas de depuración y administración de proyectos. También ayuda a diseñar interfaces, compilar el proyecto, detectar errores mientras se desarrolla, y generar el APK para instalar la app, además su integración con Gradle sirvió para manejar las dependencias y mantener el proyecto organizado.

Java

El lenguaje usado fue Java, se eligió porque es uno de los principales para desarrollar en Android y porque se trabajó en la asignatura. Con Java se implementó toda la lógica de negocio del sistema, el registro e inicio de sesión de usuarios, la gestión de gastos, las validaciones de formularios, y la comunicación con la base de datos SQLite. También pesó su estabilidad, la documentación oficial amplia y la comunidad grande de desarrolladores, esto ayudó bastante a resolver problemas durante la implementación.

SQLite

Para guardar los datos se usó SQLite, una base de datos relacional que ya viene integrada en Android, se eligió porque se necesitaba algo ligero, rápido, que funcionara sin internet. Con SQLite se guardó de forma persistente la información de usuarios y gastos, implementando las operaciones CRUD, crear, consultar, actualizar y eliminar registros. Al no depender de servidores externos, simplificó bastante el desarrollo y cumplió con lo que se necesitaba en esta versión del proyecto.

XML

Las interfaces gráficas se hicieron con XML, el lenguaje que usa Android para definir cómo se ven las pantallas, con XML se construyeron las vistas de inicio de sesión, registro de usuario, dashboard, registro de gastos y la lista de gastos. Usar XML permitió mantener separada la interfaz gráfica de la lógica en Java, esto ayudó a organizar mejor el proyecto y facilita hacer cambios en el diseño después sin afectar el funcionamiento del sistema.

Git y GitHub

Durante el proyecto se usó Git para el control de versiones y GitHub para guardar el repositorio, con esto se llevó registro de los cambios hechos, se mantuvieron copias de seguridad del código, y se documentó cómo fue avanzando el desarrollo con los commits. GitHub también ayudó a entregar el proyecto, porque el código quedó organizado y accesible, junto con el APK y la documentación que pedía la evaluación.

Arquitectura MVC

Como patrón de arquitectura se usó MVC, Modelo-Vista-Controlador, esto separó la lógica de negocio, la interfaz gráfica y el acceso a los datos, dejando el proyecto más organizado y fácil de mantener. En MoneyTrack, el Modelo es principalmente la clase DatabaseHelper y la estructura de la base de datos SQLite, la Vista son los archivos XML de las interfaces, y el Controlador son las distintas Activities que procesan las acciones del usuario y coordinan la comunicación entre la interfaz y la base de datos.

7-Arquitectura del sistema

Para desarrollar MoneyTrack se usó el patrón MVC, Modelo-Vista-Controlador. Se eligió porque permite dividir la app en componentes con responsabilidades distintas, esto facilita organizar el código, entender cómo funciona el sistema y darle mantenimiento.

Separar por responsabilidades sirve mucho en una app como MoneyTrack, hay elementos bien diferenciados, las pantallas que usa el usuario, la lógica que procesa sus acciones, y la base de datos donde se guardan los registros. Al organizar todo esto con MVC, se evita meter toda la lógica en un solo archivo, y se facilita agregar mejoras más adelante.

Patrón utilizado: Modelo–Vista–Controlador

El patrón MVC divide el sistema en tres componentes principales: Modelo, Vista y Controlador.

Modelo

El Modelo representa los datos de la aplicación y las operaciones relacionadas con su almacenamiento. En MoneyTrack, este componente está representado principalmente por la clase DatabaseHelper, la cual extiende de SQLiteOpenHelper.

Esta clase se encarga de:

- Crear y actualizar la base de datos MoneyTrack.db.
- Definir las tablas del sistema.
- Registrar usuarios.
- Validar las credenciales de inicio de sesión.
- Verificar si un correo ya está registrado.
- Obtener el nombre del usuario.
- Insertar nuevos gastos.
- Consultar los gastos almacenados.
- Actualizar los datos de un gasto.
- Eliminar gastos.
- Calcular la cantidad de gastos registrados.
- Calcular el monto total gastado.

Aunque el modelo de datos contempla las entidades Usuario, Categoría, Ingreso y Gasto, la funcionalidad principal de esta versión utiliza principalmente las tablas Usuario y Gasto. Las demás entidades permanecen definidas como parte del diseño general y pueden utilizarse en futuras ampliaciones.

Vista

La Vista corresponde a las interfaces gráficas mediante las cuales el usuario interactúa con la aplicación. En MoneyTrack, las vistas fueron desarrolladas utilizando archivos XML ubicados dentro del directorio de recursos de Android.

Entre las principales vistas se encuentran:

- activity_main.xml: pantalla de inicio de sesión.
- activity_registro_usuario.xml: formulario para crear una cuenta.
- activity_dashboard.xml: panel principal con el resumen de gastos.
- activity_registrar_gasto.xml: formulario para guardar nuevos gastos.
- activity_lista_gastos.xml: pantalla para consultar, actualizar y eliminar gastos.

Los archivos XML definen elementos como campos de texto, botones, títulos, tarjetas, colores, márgenes y distribución de los componentes. Esto permite mantener separada la presentación visual de la lógica programada en Java.

También se utilizaron archivos dentro de la carpeta drawable para mantener un diseño coherente, como fondos redondeados para botones, campos y tarjetas.

Controlador

El Controlador recibe las acciones realizadas por el usuario, ejecuta las validaciones correspondientes y se comunica con el Modelo para consultar o modificar la información.

En MoneyTrack, esta responsabilidad es asumida principalmente por las Activities:

- MainActivity: procesa el inicio de sesión, valida los campos y permite navegar hacia el registro de usuario.
- RegistroUsuarioActivity: valida la información ingresada y registra nuevos usuarios en SQLite.
- DashboardActivity: muestra el nombre del usuario, la cantidad de gastos y el total gastado, además de controlar la navegación principal.
- RegistrarGastoActivity: procesa el formulario de registro de gastos y ejecuta las validaciones del monto y la descripción.
- ListaGastosActivity: consulta los gastos y gestiona las operaciones de actualización y eliminación.

Las Activities funcionan como intermediarias entre los archivos XML y la clase DatabaseHelper. Por ejemplo, cuando el usuario presiona el botón para guardar un gasto, RegistrarGastoActivity obtiene los datos escritos en la Vista, los valida y luego llama al método insertarGasto() del Modelo.

Capas del sistema

Aunque el proyecto utiliza el patrón MVC, su funcionamiento también puede explicarse mediante tres capas principales.

Capa de presentación

Esta capa contiene las pantallas y los componentes visuales que utiliza el usuario. Está formada por los archivos XML y por los elementos gráficos ubicados en la carpeta drawable.

Su responsabilidad es presentar la información, recibir datos mediante formularios y ofrecer una navegación clara entre las distintas funcionalidades.

Capa de lógica de aplicación

Esta capa está compuesta por las Activities desarrolladas en Java. Aquí se encuentran las validaciones, la navegación, la gestión de eventos de los botones y las decisiones relacionadas con el flujo de la aplicación.

Por ejemplo, esta capa verifica que el correo tenga un formato válido, que las contraseñas coincidan, que el monto sea mayor que cero y que el usuario confirme una eliminación antes de ejecutarla.

Capa de datos

La capa de datos está representada por SQLite y por la clase DatabaseHelper. Su responsabilidad es conservar la información de manera persistente y ejecutar las operaciones solicitadas por la capa de lógica.

Esta separación permite que las pantallas no ejecuten directamente instrucciones SQL, sino que utilicen métodos específicos definidos dentro de DatabaseHelper.

Estructura de carpetas

El proyecto se encuentra organizado dentro de la estructura estándar de una aplicación Android. Los elementos principales se distribuyen de la siguiente manera:

app

```
└─ src
    └─ main
        ├── java
        │   ├── com.example.moneytrack
        │   │   ├── MainActivity.java
        │   │   ├── RegistroUsuarioActivity.java
        │   │   ├── DashboardActivity.java
        │   │   ├── RegistrarGastoActivity.java
        │   │   ├── ListaGastosActivity.java
        │   │   └─ DatabaseHelper.java
        │   └─
        ├── res
        │   ├── layout
        │   │   ├── activity_main.xml
        │   │   ├── activity_registro_usuario.xml
        │   │   ├── activity_dashboard.xml
        │   │   ├── activity_registrar_gasto.xml
        │   │   └─ activity_lista_gastos.xml
        │   └─
        ├── drawable
        │   └─ fondo_tarjeta.xml
```

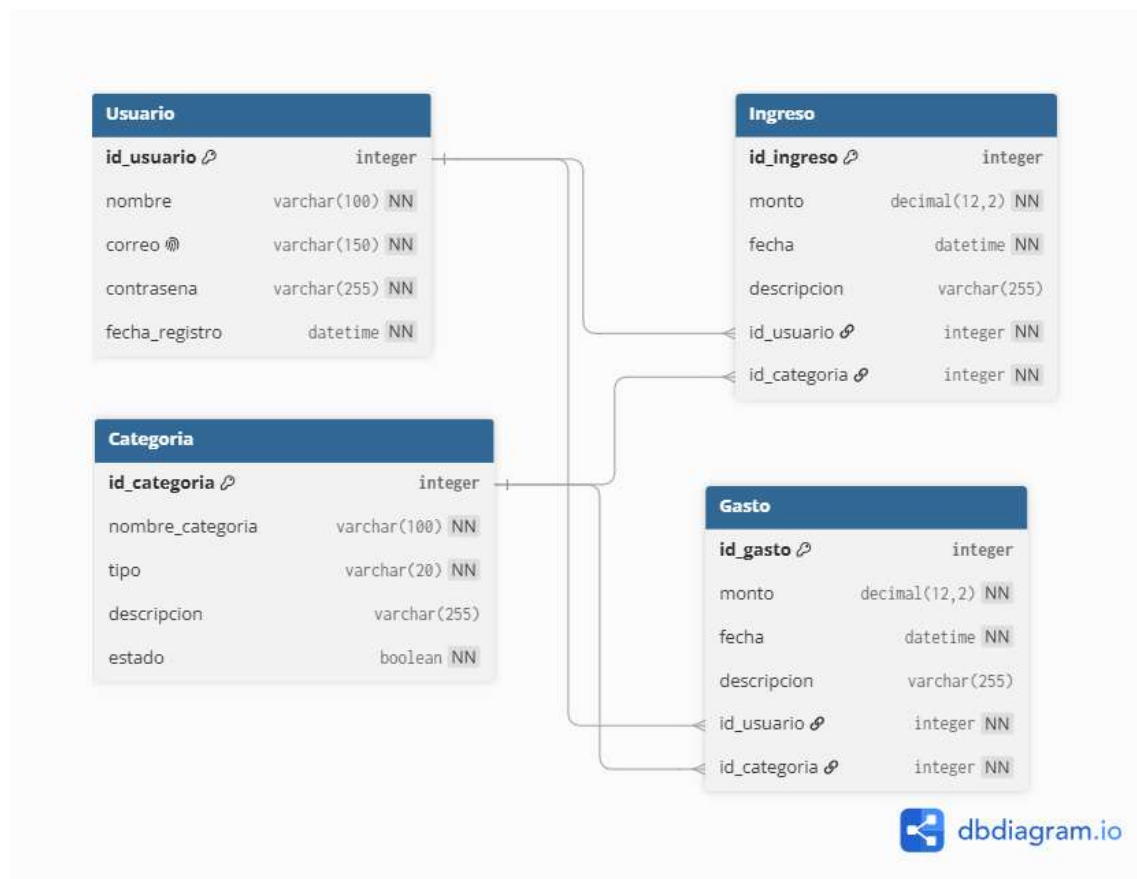
```
| | |— fondo_campo.xml
| | |— fondo_boton_principal.xml
| | |— fondo_boton_secundario.xml
| | |— fondo_boton_eliminar.xml
| |
| |— values
| |   |— colors.xml
| |   |— strings.xml
| |   |— themes.xml
|
|— AndroidManifest.xml
```

8-Modelo de datos

El modelo de datos de MoneyTrack se diseñó para representar la información necesaria para manejar usuarios y movimientos financieros personales. El diseño general tiene cuatro entidades principales, Usuario, Categoría, Ingreso y Gasto. Estas entidades guardan los datos de las personas registradas, clasifican los movimientos financieros, y mantienen un historial de ingresos y gastos.

En el diseño inicial se representó la base de datos con un Diagrama Entidad-Relación (DER). Para la app Android se usó SQLite como almacenamiento local. En la versión funcional que se desarrolló, las tablas que se usan directamente son Usuario y Gasto, porque el alcance principal se enfocó en la autenticación y en el CRUD completo de gastos. Las tablas Categoría e Ingreso se incluyeron en la estructura general para mantener coherencia con el diseño original, y para facilitar ampliaciones futuras.

Diagrama Entidad-Relación final



Diccionario de datos

Tabla Usuario

La tabla Usuario almacena las cuentas creadas dentro de la aplicación.

Campo	Tipo de dato	Restricciones	Descripción
id_usuario	INTEGER	Clave primaria, autoincremental	Identificador único de cada usuario registrado.
nombre	TEXT	No nulo	Nombre completo del usuario.
correo	TEXT	No nulo, único	Correo utilizado para identificar al usuario e iniciar sesión.
contrasena	TEXT	No nulo	Contraseña utilizada para autenticar la cuenta.
fecha_registro	TEXT	No nulo	Fecha en la que se creó la cuenta del usuario.

El campo id_usuario se genera automáticamente al insertar un registro. La restricción UNIQUE aplicada sobre correo evita cuentas duplicadas. Antes de registrar un nuevo usuario, la aplicación consulta si el correo ya se encuentra almacenado y muestra un mensaje de validación cuando existe.

Tabla Categoría

La tabla Categoría contiene la clasificación definida para los movimientos financieros.

Campo	Tipo de dato	Restricciones	Descripción
id_categoria	INTEGER	Clave primaria, autoincremental	Identificador único de la categoría.
nombre_categoria	TEXT	No nulo	Nombre asignado a la categoría.
tipo	TEXT	No nulo	Indica si la categoría corresponde a un ingreso o a un gasto.
descripcion	TEXT	Opcional	Información adicional acerca de la categoría.
estado	INTEGER	No nulo	Indica si la categoría está activa o inactiva.

El campo estado se almacena como un valor entero debido a que SQLite no posee un tipo booleano independiente. Convencionalmente, el valor 1 puede representar una categoría activa y el valor 0 una categoría inactiva.

Tabla Ingreso

La tabla Ingreso fue diseñada para almacenar las entradas de dinero del usuario.

Campo	Tipo de dato	Restricciones	Descripción
id_ingreso	INTEGER	Clave primaria, autoincremental	Identificador único del ingreso.
monto	REAL	No nulo	Cantidad de dinero recibida.
fecha	TEXT	No nulo	Fecha en la que se registró o recibió el ingreso.
descripcion	TEXT	Opcional	Información complementaria sobre el ingreso.
id_usuario	INTEGER	Clave foránea	Identifica al usuario propietario del ingreso.
id_categoria	INTEGER	Clave foránea	Identifica la categoría asignada al ingreso.

El campo id_usuario se relaciona con la clave primaria de la tabla Usuario, mientras que id_categoria se relaciona con la tabla Categoría. De esta manera, cada ingreso puede vincularse con el usuario que lo registró y con su clasificación correspondiente.

Tabla Gasto

La tabla Gasto almacena las salidas de dinero registradas desde la aplicación.

Campo	Tipo de dato	Restricciones	Descripción
id_gasto	INTEGER	Clave primaria, autoincremental	Identificador único del gasto.
monto	REAL	No nulo	Cantidad de dinero correspondiente al gasto.
fecha	TEXT	No nulo	Fecha en la que se registró el gasto.
descripcion	TEXT	Opcional	Detalle o motivo del gasto realizado.
id_usuario	INTEGER	Clave foránea, opcional en esta versión	Identifica al usuario relacionado con el gasto.
id_categoria	INTEGER	Clave foránea, opcional en esta versión	Identifica la categoría asignada al gasto.

En la versión actual, el formulario solicita el monto y una descripción, mientras que la fecha se genera automáticamente utilizando la fecha del dispositivo. Las columnas id_usuario e id_categoria forman parte de la estructura prevista en el modelo, pero todavía no se asignan al guardar los gastos. Esto se dejó como una mejora para una versión futura, en la que cada usuario podrá consultar exclusivamente sus propios movimientos y clasificarlos por categorías.

9-Descripción de pantallas

Pantalla de inicio de sesión

La pantalla de inicio de sesión es el punto de acceso principal a la app. Su función es autenticar al usuario pidiendo correo electrónico y contraseña, así solo los usuarios registrados pueden entrar a las funcionalidades del sistema.

Al presionar "Iniciar sesión", la app valida que las credenciales coincidan con lo que está guardado en la base de datos SQLite. Si la autenticación sale bien, el usuario va al Dashboard, si no, aparece un mensaje diciendo que el correo o la contraseña están mal. Desde esta misma pantalla también se puede ir al formulario de registro, con el botón "Crear cuenta".



Pantalla de registro de usuario

La pantalla de registro deja crear nuevas cuentas dentro de la app. Ahí el usuario ingresa su nombre, correo electrónico y contraseña para completar el proceso.

Antes de guardar la información, la app verifica que todos los campos obligatorios estén completos, y que el correo no esté registrado ya. Si las validaciones pasan bien, la información se guarda en SQLite y aparece un mensaje confirmando que el registro se hizo correctamente.

4:22

Crear cuenta

Complete los datos para comenzar a utilizar MoneyTrack

Nombre completo

Correo electrónico

Contraseña

Confirmar contraseña

Crear cuenta

Pantalla Dashboard

El Dashboard es la pantalla principal después de iniciar sesión. Su función es dar un resumen general de la información guardada, y servir de punto de navegación hacia las demás funcionalidades.

En esta pantalla aparece un mensaje de bienvenida con el nombre del usuario, la cantidad de gastos registrados, y el monto total gastado hasta ese momento. También tiene botones para registrar nuevos gastos, consultar la lista de gastos, y cerrar sesión de forma segura.



Pantalla Registrar gasto

La pantalla de registro de gastos permite meter nuevos movimientos financieros en la app. El usuario ingresa el monto del gasto y una descripción para identificarlo.

Al guardar, la app valida que el monto se haya ingresado bien y sea mayor que cero. Después genera automáticamente la fecha del registro, y guarda toda la información en SQLite. Si todo sale bien, aparece un mensaje confirmando que el gasto se registró con éxito.



The screenshot shows a mobile application interface for recording expenses. At the top, the status bar displays the time 4:23, signal strength, Wi-Fi, and battery level at 67%. The app's title 'Registrar gasto' is in blue, followed by the instruction 'Ingrese la información del gasto'. Below this is a white rounded rectangle containing two input fields: 'Monto' and 'Descripción (opcional)'. A purple button labeled 'Guardar gasto' is positioned at the bottom of the form. The entire screen has a light blue background, and the bottom navigation bar is visible.

Pantalla Lista de gastos

La pantalla de lista de gastos permite ver todos los gastos registrados en la app. La información se muestra con el identificador del gasto, el monto, la fecha y la descripción.

Desde esta misma pantalla el usuario puede actualizar un gasto existente, o eliminar un registro cuando ya no lo necesite. Antes de hacer estas operaciones, la app valida la información ingresada, y una vez completada la acción, actualiza automáticamente la lista para reflejar los cambios hechos en la base de datos.

The screenshot displays the 'Gastos registrados' (Registered Expenses) screen. At the top, the status bar shows the time 4:24 and signal indicators. The main content area is divided into three sections:

- Gastos registrados:** A section with the title 'Gastos registrados' and the message 'No hay gastos registrados.' (There are no registered expenses).
- Actualizar gasto:** A section with the title 'Actualizar gasto' (Update expense). It contains three input fields: 'ID del gasto' (Expense ID), 'Nuevo monto' (New amount), and 'Nueva descripción' (New description). Below these fields is a purple button labeled 'Actualizar gasto'.
- Eliminar gasto:** A section with the title 'Eliminar gasto' (Delete expense). It contains one input field: 'ID del gasto'. Below this field is a purple button labeled 'Eliminar gasto'.

The bottom of the screen shows the standard Android navigation bar with back, home, and recent apps buttons.

10-Decisiones de diseño

Durante el desarrollo de MoneyTrack se tomaron varias decisiones técnicas y funcionales, la idea era mantener la app sencilla, estable, y acorde al alcance del proyecto. Estas decisiones se basaron en lo que pedía la asignatura, el tiempo disponible, las tecnologías que ya se dominaban, y la necesidad de entregar una app móvil completamente funcional.

Uso de Android Studio como entorno de desarrollo

Se usó Android Studio como entorno principal porque es la herramienta oficial para desarrollar en Android. Con esto fue más fácil crear las Activities, diseñar interfaces, compilar el proyecto, correrlo en emuladores, y generar el APK.

Se descartaron otras opciones como .NET MAUI o editores más generales, eran más complejas para lo que necesitaba el proyecto y podían alargar el tiempo de desarrollo. Android Studio se integraba mejor con Java, XML y SQLite.

Uso de Java como lenguaje principal

Se eligió Java porque era el lenguaje que se trabajó en la asignatura, y porque tiene compatibilidad completa con Android nativo. Con esto se pudo implementar la lógica de autenticación, las validaciones, la navegación, las operaciones CRUD, sin meter tecnologías nuevas que complicaran el proyecto. Se descartó usar Kotlin en esta versión, aunque podría considerarse más adelante.

Uso de SQLite como base de datos local

Se eligió SQLite porque permite guardar información directo en el dispositivo, sin depender de internet ni de servidores externos. Esto encajaba bien con lo que MoneyTrack necesitaba, registrar usuarios y gastos localmente, y cumplía con el requisito de persistencia y el CRUD completo que pedía la consigna.

Se descartaron opciones como Firebase, Supabase o conectarse directo a SQL Server, necesitaban configuración adicional, conectividad, y en algunos casos una API intermedia. También se descartó Room, para mantener todo alineado con SQLite y las tecnologías que ya se estaban usando.

Uso del patrón MVC

Se usó el patrón MVC para separar la interfaz, la lógica y el acceso a los datos. Los XML son la Vista, las Activities cumplen de Controlador, y la clase DatabaseHelper junto con SQLite son el Modelo. Esto ayudó a mantener una estructura comprensible y facilitó corregir errores durante el desarrollo.

Se descartaron arquitecturas más complejas como MVVM o Clean Architecture, para el alcance académico de MoneyTrack hubieran añadido complejidad innecesaria.

Priorización del CRUD de gastos

El CRUD de Gasto se definió como la funcionalidad principal, la consigna exigía al menos un conjunto completo de operaciones CRUD, y el registro de gastos es el núcleo de MoneyTrack. Se implementaron las operaciones de crear, consultar, actualizar y eliminar gastos, con validaciones y confirmación antes de eliminar.

Se descartó hacer un CRUD completo de ingresos y categorías en esta versión, hubiera ampliado demasiado el alcance y aumentado el riesgo de no terminar bien la funcionalidad principal.

Simplificación del formulario de gastos

Al principio se pensó en incluir un campo de categoría en el formulario de gastos, pero se decidió quitarlo para mantener el flujo más simple y evitar inconsistencias entre la interfaz y la base de datos.

El formulario final pide solo monto y descripción, la fecha se genera automáticamente. Esto redujo los campos obligatorios y permitió enfocarse en que el CRUD funcionara de forma estable. La gestión de categorías quedó como mejora futura.

Registro e inicio de sesión local

Se decidió meter registro e inicio de sesión usando la tabla Usuario de SQLite, así la app tiene un flujo más completo y la tabla de usuarios se usa de verdad. También se añadió la restricción de correo único y validación de credenciales. Se descartó una autenticación remota, no era necesaria para esta versión y hubiera requerido servicios externos.

Dashboard con datos reales

El Dashboard se diseñó para mostrar información sacada directamente de SQLite, como la cantidad de gastos registrados y el total gastado. Se evitó hacer un Dashboard solo decorativo, mostrar datos reales mejora la utilidad de la app y demuestra que la interfaz y la base de datos están bien integradas.

Se descartaron gráficos, reportes visuales y estadísticas avanzadas, no eran indispensables para cumplir con la funcionalidad principal.

Diseño visual coherente

Se mantuvo una identidad visual uniforme en todas las pantallas, colores consistentes, fondos claros, tarjetas, campos redondeados, botones con estilos

parecidos. Esto se hizo para mejorar la experiencia del usuario y cumplir con el criterio de diseño móvil de la rúbrica.

Se descartó usar librerías externas de diseño o animaciones avanzadas, para evitar dependencias innecesarias y mantener la estabilidad del proyecto.

Actualización y eliminación por ID

La pantalla de lista permite actualizar y eliminar gastos usando el identificador del registro. Se eligió esta solución porque era sencilla de implementar y dejaba completar el CRUD de forma estable.

Se descartó, por ahora, usar un RecyclerView con botones individuales o una pantalla de detalle por cada gasto, eso implicaba modificar bastante la estructura actual. Esta mejora podría meterse en una versión futura para facilitar la interacción.

Almacenamiento local y funcionamiento sin Internet

La app se diseñó para funcionar completamente sin conexión a internet, esto mejora la accesibilidad y evita que el usuario dependa de servicios externos. Por eso los datos se quedan en el dispositivo donde se instala la app. Se descartó la sincronización entre dispositivos, las copias de seguridad automáticas y el almacenamiento en la nube, para mantener el alcance controlado.

Seguridad de la contraseña

Para esta versión académica, la contraseña se guarda directo en la base de datos local, así se puede validar el inicio de sesión. Esto simplificó la implementación, aunque se reconoce que en una app de producción haría falta aplicar seguridad adicional, como cifrado o hash. Esa mejora quedó fuera del alcance de esta versión.

Conclusión de las decisiones de diseño

Estas decisiones permitieron construir una app funcional, comprensible, adecuada al tiempo disponible. En vez de ampliar el alcance con varios módulos incompletos, se priorizó una funcionalidad central estable, un flujo completo de usuario, persistencia local, validaciones, y una interfaz coherente.

Las funcionalidades que se descartaron no están eliminadas para siempre, quedan como posibles mejoras para futuras versiones de MoneyTrack.

11-Plan y resultados de pruebas

Para asegurar que MoneyTrack funcionara bien, se diseñó un plan de pruebas enfocado en verificar las funcionalidades principales que se implementaron durante el desarrollo. Las pruebas se fueron haciendo poco a poco, conforme se completaban los distintos módulos de la app, esto ayudó a detectar errores y corregirlos antes de la integración final.

Se probó usando el emulador de Android Studio y un dispositivo Android, revisando tanto el comportamiento de la interfaz como la interacción con la base de datos SQLite. Se evaluó la autenticación, el registro de usuarios, la navegación entre pantallas, y las operaciones CRUD sobre los gastos registrados.

Plan de pruebas

Las pruebas funcionales se planificaron considerando los procesos más importantes que realiza un usuario dentro de la aplicación. Entre los aspectos evaluados se encuentran:

- Registro de nuevos usuarios.
- Inicio de sesión con credenciales válidas e inválidas.
- Validación de campos obligatorios en los formularios.
- Registro de nuevos gastos.
- Consulta de la lista de gastos almacenados.
- Actualización de la información de un gasto.
- Eliminación de gastos registrados.
- Actualización automática del Dashboard después de cada operación.
- Navegación entre todas las pantallas de la aplicación.
- Persistencia de la información almacenada en SQLite.

Cada prueba fue ejecutada varias veces para verificar que el comportamiento fuera consistente y que la aplicación mantuviera la integridad de la información almacenada.

Resultados obtenidos

Los resultados fueron satisfactorios, todas las funcionalidades principales de MoneyTrack respondieron bien después de aplicar las correcciones durante el desarrollo.

Se verificó que el sistema registra bien a los nuevos usuarios, valida el inicio de sesión usando SQLite, y restringe el acceso cuando las credenciales no son válidas. El CRUD de gastos también funciona bien, permite crear, consultar, actualizar y eliminar registros correctamente, la información se mantiene guardada incluso después de cerrar la app.

El Dashboard también se validó, se comprobó que muestra bien el nombre del usuario autenticado, la cantidad de gastos registrados, y el monto total gastado, actualizando esta información conforme se hacen nuevas operaciones.

En general, la app se comportó de forma estable y cumplió con los objetivos funcionales definidos para esta versión.

Errores encontrados y correcciones aplicadas

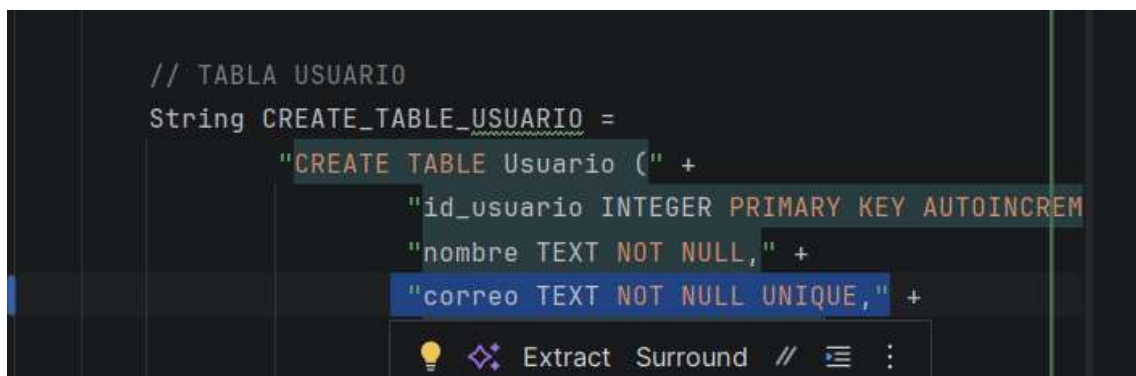
Durante el desarrollo del proyecto se identificaron diversos inconvenientes técnicos que fueron corregidos antes de finalizar la aplicación. Los principales fueron los siguientes:

Error en el registro de usuarios

Inicialmente la aplicación permitía registrar múltiples usuarios utilizando el mismo correo electrónico.

Corrección aplicada: Se agregó una restricción **UNIQUE** al campo correo en la tabla **Usuario** y se implementó una validación previa para impedir registros duplicados.

```
// TABLA USUARIO
String CREATE_TABLE_USUARIO =
    "CREATE TABLE Usuario (" +
        "id_usuario INTEGER PRIMARY KEY AUTOINCREM" +
        "nombre TEXT NOT NULL," +
        "correo TEXT NOT NULL UNIQUE," +
```

A screenshot of a code editor showing SQL code for creating a table named 'Usuario'. The code is written in a dark-themed editor with syntax highlighting. The code defines a table with three columns: 'id_usuario' (INTEGER PRIMARY KEY AUTOINCREM), 'nombre' (TEXT NOT NULL), and 'correo' (TEXT NOT NULL UNIQUE). The code is wrapped in a String variable named CREATE_TABLE_USUARIO. The 'correo' field is highlighted with a blue background. Below the code, there is a toolbar with icons for lightbulb, diamond, Extract, Surround, and other editing functions.

Error en el inicio de sesión

En una versión inicial, el botón **Iniciar sesión** permitía acceder al Dashboard sin verificar las credenciales ingresadas.

Corrección aplicada: Se implementó el método validarUsuario() en DatabaseHelper, permitiendo comprobar el correo y la contraseña almacenados en SQLite antes de conceder el acceso.

```

public boolean validarUsuario(String correo, String contraseña) {
    SQLiteDatabase db = this.getReadableDatabase();

    Cursor cursor = db.rawQuery(
        "SELECT * FROM Usuario WHERE correo = ? AND contraseña = ?",
        new String[]{correo, contraseña}
    );

    boolean existe = cursor.getCount() > 0;

    cursor.close();

    return existe;
}

```

Error durante el registro de gastos

Al principio, el formulario permitía intentar registrar gastos sin completar correctamente la información requerida.

Corrección aplicada: Se añadieron validaciones para comprobar que el monto fuera obligatorio y mayor que cero antes de guardar el registro.

```

if (montoTexto.isEmpty()) {
    etMonto.setError("Debe ingresar un monto");
    etMonto.requestFocus();
    return;
}

double monto;

try {
    monto = Double.parseDouble(montoTexto);
} catch (NumberFormatException e) {
    etMonto.setError("Ingrese un monto válido");
    etMonto.requestFocus();
    return;
}

if (monto <= 0) {
    etMonto.setError("El monto debe ser mayor que cero");
    etMonto.requestFocus();
    return;
}

```


Error de actualización y eliminación

Durante las primeras pruebas se detectaron inconvenientes relacionados con la actualización y eliminación de registros cuando el identificador ingresado no existía.

Corrección aplicada: Se incorporaron validaciones para verificar la existencia del registro y mostrar mensajes informativos al usuario cuando el ID no era válido.

```
if (idTexto.isEmpty()) {
    etIdEliminar.setError("Ingrese el ID del gasto");
    etIdEliminar.requestFocus();
    return;
}

int idGasto;

try {
    idGasto = Integer.parseInt(idTexto);
} catch (NumberFormatException e) {
    etIdEliminar.setError("Ingrese un ID válido");
    etIdEliminar.requestFocus();
    return;
}

if (idGasto <= 0) {
    etIdEliminar.setError("El ID debe ser mayor que cero");
    etIdEliminar.requestFocus();
    return;
}

mostrarConfirmacionEliminacion(idGasto);
}
```

```
if (eliminado) {  
  
    Toast.makeText(  
        context: this,  
        text: "Gasto eliminado correctamente",  
        Toast.LENGTH_SHORT  
    ).show();  
  
    etIdEliminar.setText("");  
  
    mostrarGastos();  
  
} else {  
  
    Toast.makeText(  
        context: this,  
        text: "No existe un gasto con ese ID",  
        Toast.LENGTH_SHORT  
    ).show();  
  
}  
  
}
```

Error en la navegación entre pantallas

Se presentaron algunos problemas relacionados con la creación de nuevas Activities y la navegación dentro de la aplicación.

Corrección aplicada: Se ajustó el archivo AndroidManifest.xml y se verificó la correcta comunicación mediante Intent entre todas las pantallas implementadas.

```
btnRegistrarGasto.setOnClickListener( View v -> {
    Intent intent = new Intent(
        packageContext: DashboardActivity.this,
        RegistrarGastoActivity.class
    );
    startActivity(intent);
});

btnVerGastos.setOnClickListener( View v -> {
    Intent intent = new Intent(
        packageContext: DashboardActivity.this,
        ListaGastosActivity.class
    );
    startActivity(intent);
});

btnCerrarSesion.setOnClickListener( View v -> {

    Intent intent = new Intent(
        packageContext: DashboardActivity.this,
        MainActivity.class
    );

    intent.setFlags(
        Intent.FLAG_ACTIVITY_NEW_TASK |
        Intent.FLAG_ACTIVITY_CLEAR_TASK
    );

    startActivity(intent);
});
```

Error en la creación de actividades

Durante la implementación de la pantalla de registro aparecieron errores relacionados con la plantilla generada automáticamente por Android Studio, específicamente con el identificador main y componentes de Edge-to-Edge que no existían en el diseño.

Corrección aplicada: Se eliminó el código innecesario generado automáticamente y se simplificó la Activity para adaptarla al diseño utilizado en el proyecto.

Error de compilación

En distintas etapas del desarrollo surgieron errores ocasionados por declaraciones duplicadas de package e import, así como por configuraciones propias del proyecto.

Corrección aplicada: Se reorganizaron las importaciones, se eliminaron declaraciones repetidas y se verificó la estructura de cada archivo antes de continuar con el desarrollo.

Conclusión de las pruebas

Las pruebas realizadas permitieron comprobar que **MoneyTrack** cumple correctamente con las funcionalidades previstas para esta primera versión. La aplicación presentó un funcionamiento estable durante las pruebas finales, manteniendo la integridad de la información almacenada y ofreciendo una navegación fluida entre las diferentes pantallas.

Los errores detectados durante el desarrollo fueron corregidos oportunamente, lo que permitió entregar una aplicación funcional con autenticación de usuarios, persistencia local mediante SQLite y un CRUD completo para la gestión de gastos personales. Los casos de prueba detallados y sus evidencias se presentan en el apartado correspondiente de la documentación.

Conclusión:

Con MoneyTrack se cumplió el objetivo principal del proyecto. Diseñar e implementar una app móvil para Android que registre y gestione gastos personales, guardando todo en SQLite. El resultado fue una app funcional. Tiene sistema de registro e inicio de sesión, un Dashboard con información resumida, un CRUD completo para manejar gastos. Cumple con lo que pedía el proyecto práctico final.

Durante el desarrollo pude aplicar lo aprendido en la carrera. Programación orientada a objetos, desarrollo de apps móviles, bases de datos relacionales, diseño de interfaces, arquitectura de software. El proyecto también me ayudó a fortalecer otras habilidades. Resolver problemas, depurar código, implementar validaciones, usar herramientas de control de versiones, escribir documentación técnica.

Aprendí que planificar bien antes de programar es fundamental. Definir el modelo de datos, la arquitectura del sistema, el alcance del proyecto. Esto mantuvo todo organizado y facilitó meter nuevas funcionalidades sobre la marcha. Las pruebas en cada etapa ayudaron mucho. Se detectaban y corregían errores a tiempo, mejorando la estabilidad de la app.

Si el proyecto siguiera creciendo, le metería más funcionalidades. Gestión completa de ingresos y categorías, reportes y gráficos estadísticos, manejo de presupuestos, exportar información, sincronizar datos con la nube. Todo esto ampliaría lo que MoneyTrack puede hacer, daría una experiencia más completa al usuario.

MoneyTrack es una solución funcional para registrar y controlar gastos personales. Muestra también la aplicación práctica de lo aprendido en la carrera de Ingeniería de Software. El proyecto unió análisis, diseño, desarrollo, pruebas y documentación en un mismo proceso. El resultado cumple los objetivos planteados, y deja una base sólida para futuras mejoras.

Bibliografía:

Develop for Android. (s. f.-b). Android Developers. <https://developer.android.com/develop>

Learn Java - dev.java. (s. f.). Dev.java: The Destination For Java Developers.

<https://dev.java/learn/>

SQLite Documentation. (s. f.-b). <https://sqlite.org/docs.html>

GitHub Documentación de ayuda de .com. (s. f.). GitHub Docs. <https://docs.github.com/es>